

Norwegian Singles Approach (NSA) Calculator

Project Report

Martin Styve Pedersen
INF221 Advanced Functional Programming @ UiB

Spring 2026

`nsa.martinstyve.no`

1 Introduction

Norwegian Singles Approach (NSA) is a web application where you can input a running race time and distance, and the application will parse the input and parse it to a VDOT number. VDOT is a generalized numerical value which represents your fitness. The web application uses the VDOT to estimate how fast you will run at other events, and suggests a pace range for running interval training, for different types of intervals. The application also has a short information card with description of running methodology.

2 Description of project

2.1 initial idea / background

My motivation behind this project is that this is a personal need I have. I am currently using `lactrace.com` but think the site has too much noise and other features. I only wanted these simple features, so I wanted to make a simpler and more focused site for this purpose.

2.2 goals of the project

A safe and reliable site for this particular need. The project should accept a race time and distance as input, compute a VDOT value, use that value to predict equivalent performances for standard race distances, and calculate interval pace ranges. It should also stay minimal, reliable, and easy to use, so that the site focuses on the specific functionality I need without unnecessary extra features.

2.3 What the project does

When a user visits the application, they are presented with a homepage with an input form. The user enters their recent race time and selects a race distance from a dropdown menu of

common race distances, or chooses to enter a custom distance. After submitting the form, the application displays a result page showing the VDOT value, equivalent race times for standard distances, and recommended pace ranges for different interval training zones. If the user enters invalid input, E.g time in wrong format or a value out of range (like a negative number), the application displays a clear error message on the homepage, and allows user to resubmit an input.

2.4 How the project turned out

The project has turned out well and matches my original goals, extended to focus more on equivalent race times. The application successfully does what I intended it to do. The simplicity of the interface and reliability of the calculations work particularly well. There is a limitation with the VDOT formula and interval pace ranges for really fast and slow times. So I have set the limits of the application to VDOT equivalent of a 3:20 1500m (WR is 3:26) and 7 hour marathon (regular cut-off time I believe) which seems reasonable.

2.5 Process

I developed this project end to end, starting with the core VDOT formula and iteratively building out parsing, safety checks, and an HTTP wrapper. I wanted to create small, working MVPs and I incrementally expanded the project and did necessary refactoring,

A technical hurdle I encountered early was with the “predictive” side of the calculator. While calculating VDOT from a time is straightforward, predicting a race time from a VDOT is mathematically complex because the formula isn’t easily reversible. I solved this by implementing a numerical inversion using a binary search to find the time that produces the target VDOT.

```
bisect :: (VDOT -> VDOT) -> VDOT -> VDOT -> VDOT -> VDOT
bisect f target low high
  | (high - low) < 0.01 = mid
  | f mid > target = bisect f target mid high
  | otherwise = bisect f target low mid
where
  mid = (low + high) / 2

equivalentTime :: VDOT -> RaceDistance -> Either VDOTError Int
equivalentTime vdot distance
  | roundedTime < minLimit = Left TimeOutOfRange
  | roundedTime > maxLimit = Left TimeOutOfRange
  | otherwise              = Right roundedTime
where
  calculatedTime = bisect ('calculateVDOT' distance) vdot 199 25201
  roundedTime    = round calculatedTime
  minLimit = 200           -- 00:03:20
  maxLimit = 25200         -- 07:00:00
```

As the project progressed, I replaced generic placeholders with concrete ADTs like `RunTime` and `RaceDistance`.

```
buildResultPage :: RT.RunTime -> RD.RaceDistance -> Either AppError (
  Html ())
```

And I tightened the logic with `Either` based error handling and ensured reliability through QuickCheck properties, specifically testing “round-trip” invariants.

```
prop_secondsToRunTime_Inversion :: NonNegative Int -> Property
prop_secondsToRunTime_Inversion (NonNegative totalSeconds) =
  runTimeToSec (secToRunTime totalSeconds) === totalSeconds
```

2.6 Future Extensions

One natural extension would be to add support for cycling by using functional threshold power (FTP) instead of VDOT, which would allow cyclists to use the same training guidance concept adapted to watt outputs and training zones. Other realistic improvements could include support for more workout types, better styling and visualizations, and saved user profiles for returning users.

3 How to use the project

3.1 Build and run

- clone the repository
- run `cabal build`
- run `cabal run`
- open `http://localhost:6767`
- test with `cabal test`
- prerequisites: glasgow haskell compiler (ghc)

3.2 Using the web app

The application requires a race time and a race distance as input. The time can be entered in standard formats such as `mm:ss` or `h:mm:ss`, and the distance can either be selected from a list of preset race distances or entered manually as a custom distance in meters. After submitting the form, the site calculates the VDOT value and presents equivalent race times and recommended interval paces on the result page. If the input is invalid or incomplete, the user is shown a clear error message so they can correct the values and try again.

Norwegian Singles Running Calculator

Calculate optimal training paces using the Norwegian Singles method

Time

Race distance

1500 m

Custom distance (meters)

only used when Custom distance is selected from dropdown

Analyze

Figure 1: The application homepage with input form.

Norwegian Singles Running Calculator

Calculate optimal training paces using the Norwegian Singles method

expected seconds 00-59

Time

Race distance

1500 m

Custom distance (meters)

only used when Custom distance is selected from dropdown

Analyze

Figure 2: Clear error feedback for invalid time formats.

Result

VDOT: 48.9

Equivalent Race Performances

Distance	Estimated Time
1500 m	5:30
1 mile	5:57
3000 m	11:46
5 km	20:18
10 km	42:06
10 mile	1:09:48
Half marathon	1:33:14
Marathon	3:14:09

Interval Pace Ranges

Zone	Pace range	Intensity
Short Intervals	4:19 - 4:27 /km	15k intensity
Medium Intervals	4:25 - 4:33 /km	HM intensity
Long Intervals	4:31 - 4:36 /km	30k intensity

[Go Back](#)

Figure 3: The result page showing calculated VDOT and training zones.

4 Functional Programming Techniques Applied

4.1 Algebraic data types and domain modeling

4.1.1 Theory

Algebraic data types (ADTs) model domain concepts using sum and product types. Sum types let you list distinct alternatives, like a distance preset or a custom distance, while product types combine fields into a single value. Modeling domain concepts as concrete types gives the compiler and reader a clear contract. Functions operate on well-defined values instead of trivial strings or numbers, and pattern matching forces you to handle each case explicitly. This helps make invalid states harder to represent and mistakes easier to catch.

4.1.2 In code

```
data RunTime
  = MS Int Int
  | HMS Int Int Int

data RaceDistance
  = FifteenHundred
  | OneMile
  | ThreeK
  | FiveK
  | TenK
  | TenMile
  | HalfMarathon
  | Marathon
  | CustomDistance Double
```

4.1.3 How I applied it

I modeled central concepts (`RunTime`, `RaceDistance`, `InputError`, `VDOTError`, `AppError`) as ADTs rather than passing around strings and numbers.

4.2 Monadic parsing combinators (Megaparsec)

4.2.1 Theory

Parser combinators let you build complex parsers from small, reusable pieces, making it far easier to handle structured input than by manually splitting text. In Haskell, parser types are compositional and work naturally with monadic sequencing (using `do` notation) and applicative operators (like `<$>` and `<|>`) to express alternatives and transformations clearly. This approach makes the input rules explicit and easy to extend, while also providing good error reporting when parsing fails.

4.2.2 In code

```
timeParser :: Parser RunTime
timeParser = do
  parts <- digits 'sepBy1' char ':'
  case parts of
    [m, s] -> if s < 60 then pure (MS m s) else customFailure
      InvalidSeconds
    [h, m, s] ->
      if m >= 60 then customFailure InvalidMinutes
      else if s >= 60 then customFailure InvalidSeconds
      else pure (HMS h m s)
    _ -> customFailure InvalidFormat
```

4.2.3 How I applied it

I used Megaparsec to parse time formats and distance choices while validating constraints (seconds under 60, valid custom distance range).

4.3 Typed error handling with Either and error algebra

4.3.1 Theory

Either *e a* models computations that may fail, with `Left e` for failure and `Right a` for success. This is better than crashing the program or throwing uncaught exceptions, because the compiler can help track errors and the code makes it clear what can go wrong. Error algebra means defining domain specific error types (like `InputError`, `VDOTError`) and composing them across layers, so each layer can handle or forward errors properly.

4.3.2 In code

```
data AppError
  = MissingRequiredInput
  | InputParseError InputError
  | VdotCalculationError VDOTError

parseTime :: Text -> Either InputError RunTime
parseTime = either (Left . bundleToInputError InvalidFormat) Right .
  runParser timeParser ""
```

4.3.3 How I applied it

Parsing, VDOT prediction, and request validation all return typed errors. The web layer converts these errors into user-facing messages.

4.4 Higher-order functions and partial application

4.4.1 Theory

Higher-order functions either take functions as arguments or return functions, treating functions as first class values just like any other data. Partial application is the technique of supplying fewer arguments than a function expects, creating a new specialized function, enabling concise composition and reuse. Together, these ideas let you write general functions that become specific and composable when partially applied.

4.4.2 In code

```
calculatePaces vdot = map (calculateZonePace vdot) [minBound ..
    maxBound]

calculatedTime = bisect ('calculateVDOT' distance) vdot 199 25201
```

4.4.3 How I applied it

I reused generic functions by partially applying project specific parameters (for example, fixed vdot or fixed distance).

4.5 Recursion for numerical methods (bisection)

4.5.1 Theory

Recursion in functional code expresses repeated advancement naturally, as a function can call itself with clarified inputs until a termination condition is met. Bisection is a classic algorithm that repeatedly halves an interval, evaluating the function at the midpoint to decide which half contains the solution. This approach is deterministic and pure, making it ideal for numerical computations in a functional setting.

4.5.2 In code

```
bisect :: (VDOT -> VDOT) -> VDOT -> VDOT -> VDOT -> VDOT
bisect f target low high
  | (high - low) < 0.01 = mid
  | f mid > target = bisect f target mid high
  | otherwise = bisect f target low mid
where
  mid = (low + high) / 2
```

4.5.3 How I applied it

Because the VDOT formula is not trivially invertible, I implemented recursive bisection to estimate the equivalent race time from a target VDOT. To reinforce the domain logic, I defined `bisect` to operate specifically on the `VDOT` wrapper rather than generic `Double` values, as the function is only intended for these calculations.

4.6 Finite domain modeling with `Enum` and `Bounded`

4.6.1 Theory

For closed domains (sets with a fixed, finite number of values), deriving `Enum` and `Bounded` gives safe, total enumeration of all constructors. This lets you iterate over all cases automatically without manual string lists or magic numbers. The type system ensures you cannot accidentally miss a case or add an invalid value outside the domain.

4.6.2 In code

```
data Zone
  = ShortRep
  | MediumRep
  | LongRep
  deriving (Show, Eq, Enum, Bounded)

calculatePaces vdot = map (calculateZonePace vdot) [minBound ..
  maxBound]
```

4.6.3 How I applied it

I represented interval training zones as a finite type and generated all zones programmatically rather than maintaining manual string lists.

4.7 Testing with `HUnit` and `QuickCheck`

4.7.1 Theory

Unit tests verify concrete examples with specific inputs and expected outputs, documenting exact behavior with property based testing with `QuickCheck` asserts general laws and invariants that must hold over many randomly generated inputs. Properties are especially useful for mathematical or reversible functions, as they reveal edge cases that a few hand-written unit tests might miss. Together, these approaches provide both precision and validation.

4.7.2 In code

```
testCase "showRunTime formats sub one minute" $
  RT.showRunTime (secToRunTime 59) @?= pack "0:59"

prop_secondsToRunTime_Inversion :: NonNegative Int -> Property
prop_secondsToRunTime_Inversion (NonNegative totalSeconds) =
  runTimeToSec (secToRunTime totalSeconds) === totalSeconds
```

4.7.3 How I applied it

I used HUnit for exact behavior checks (formatting/parsing examples) and QuickCheck for invariants (roundtrip and conversion properties).

4.8 Conclusion Functional Programming Techniques

Overall, the techniques above complemented each other well. ADTs constrained the domain, parser combinators and `Either` made input handling robust, recursion and higher-order functions kept the core calculations modular, and mixed testing strategies validated both specific behaviors and general laws. For this project, functional techniques were not only relevant to meet the requirements of the assignment, but practically useful for correctness, maintainability, and predictable behavior.

5 Self evaluation

5.1 What was positive about working on this project?

A positive thing about working on this project is that it gave me the chance to put the functional programming techniques from the course into practice in a real project I was personally motivated to create. It was especially useful to work with ideas from the practical sessions and lectures, like monadic parsing and QuickCheck testing. The project gave me a better understanding of how techniques like monads, parsing, and typed errors can make code more structured and easier to reason about.

5.2 What would I have done differently if I were to do it again?

If I were to do it again, I would probably build the frontend with plain html, css, and JavaScript instead of generating the html with Lucid, because that felt a bit cumbersome. I would also be stricter about using algebraic data types throughout the project, instead of relying on integers and text and converting them later on. In hindsight, modeling the domain directly with types early on would have saved me from multiple rounds of small refactors.